Unbounded reliability from composable textual verification

A task-independent runtime using bounded model inputs

Research note accompanying Lecture 17 — 1 November 2026

Introduction

This research note studies reliable AI systems for well-defined problems whose requests are expressed in natural language. Learning is crucial because learned models are currently the only general tool available for processing such requests and producing answers across broad domains. Learning systems, however, are normally trained and evaluated through statistical criteria such as average loss or benchmark accuracy. Excellent statistical performance can coexist with incorrect answers on individual requests. We therefore require the system to retain learned components and their natural-language capabilities while eliminating incorrect returned answers.

As a second requirement, we pursue a less ambitious learning target than correctness on unrestricted problems. We assume that one learned component can answer every well-formed verification question whose textual representation has length at most a fixed bound. Achieving such bounded reliability remains highly nontrivial, but for a fixed finite alphabet the target concerns a finite set of inputs and is strictly weaker than unrestricted reliability. We construct a task-independent *runtime*, an ordinary software program with no natural-language understanding capabilities of its own, that uses these bounded judgments to check answers to problems of unbounded size. Assuming the bounded judgments are correct, the runtime is reliable for every generator. Because reliability is trivial for a system that never answers, we also study coverage. With an appropriate generator, the construction covers every request whose correct answer has a finite certificate over a finite sound rule library. In particular, every computably enumerable correctness relation admits such a representation.

The core idea of the construction is as follows. We require a generator to propose an answer together with a finite *answer certificate* explaining why the answer is correct. A learned verifier judges the bounded textual rules used in the certificate. The runtime checks each application of those rules through exact references, substitutions, and string comparisons. The runtime returns the proposed answer only if the mechanically constructed final claim says verbatim that the proposed answer correctly answers the exact original request. If every rule accepted by the verifier preserves truth, every answer returned by the runtime is correct.

Certificates and mechanical checking have a long history. Finite rule systems can express general computation, while proof systems and proof-carrying code separate the production of a solution from mechanical checking of its certificate [1, 3, 4]. Recent prover–verifier and debate methods apply related ideas to learned systems [7, 8, 9, 10]. We combine these lines of work in a setting where requests, intermediate assertions, and rules may remain largely in natural language and the learned verifier is required to be reliable only on bounded textual inputs.

Long answers and long requests create additional problems because the relevant text may not fit in one bounded model input. Consider first the case of long answers. For example, the request may ask for a long list of desk-tidying instructions. The runtime can request judgments of the short items one by one and of a short general rule that connects those judgments to the correctness of the complete output. The runtime then applies the rule to the proposed answer by mechanically matching its premises against statements established earlier about the request and proposed answer. Nothing in the runtime is specific to lists: the model output expresses the task-specific reasoning through statements and implications. The same generic arrangement can therefore handle a wide variety of complex tasks and long responses using model inputs of bounded length.

A long request presents a harder problem because no such bounded model input can contain the entire request, yet determining what counts as a correct answer may require combining information from all its parts. After proving the certificate and runtime theorems, we show how a generator can propose bounded rules that organize the input. The runtime uses generic string operations to check that the resulting certificate accounts for the exact complete request.

Section 1 defines correctness. Section 2 gives the certificate theorem independently of any runtime, and Section 3 explains the computational scope of finite rule libraries. Sections 4 to 6 then give the executable protocol and its guarantee. Section 7 first applies the runtime to short requests and identifies the steps that cease to fit in bounded verifier calls. Section 8 develops the general long-input method, and Section 9 gives complete arithmetic and tax constructions. The implementation and complete traces accompany this note.

1 The reliability objective and the input model

This section defines requests, correct answers, reliability, and completeness. At a high level, the problem is as follows: The user enters *requests* and expects *answers*. Fix a finite alphabet Σ with at least two symbols and let $\mathcal{S} = \Sigma^*$ be the set of finite strings over Σ, which we also call texts. Requests and answers are elements of $\mathcal{S}$. We define correctness of answers abstractly, as a binary relation Q on $\mathcal{S}$:

$$Q \subseteq \mathcal{S} \times \mathcal{S}\,.$$

Notably, the domain,

$$\mathrm{dom}(Q) = \{x \in \mathcal{S} : \exists y \in \mathcal{S} \text{ s.t. } (x, y) \in Q\}$$

of Q consists of those texts for which a well-defined answer exists. Every $y \in \mathcal{S}$ has the correct type to serve as an answer; Q determines whether y correctly answers a given request. Besides postulating the existence of Q, the semantics of correctness is otherwise left undefined. This is a strength: The results do not depend on the exact details of what goes and does not go into Q (more about this later). In what follows we will call Q the *correctness relation*.

In addition to Q, we also need a subset $\mathcal{T} \subseteq \mathcal{S}$ of all finite strings, which we call the *set of true assertions*. Intuitively, for the set of true assertions and the correctness relation to be consistent with each other, assuming as an example that the targeted natural language is English, one expects that for any $(x, y) \in \mathcal{S} \times \mathcal{S}$, y is a correct answer to x if and only if the text

$$\mathrm{Goal}(x, y) = \texttt{The response} \parallel \mathrm{quote}(y) \parallel \texttt{correctly answers the request} \parallel \mathrm{quote}(x) \parallel . \quad (1)$$

is a true assertion. In short,

$$\mathcal{T} \cap \mathrm{range}(\mathrm{Goal}) = \mathrm{Goal}(Q) \quad (2)$$

needs to hold. Here range is the range of a map and $\mathrm{Goal}(Q) = \{\mathrm{Goal}(x, y) : (x, y) \in Q\}$.

In Eq. (1), $\parallel$ stands for concatenating the strings on its sides and $\mathrm{quote} : \mathcal{S} \to \mathcal{S}$ is a map that "quotes" its argument in such a way that there is a partial decoding map $\mathrm{decquote} : \mathcal{S} \rightharpoonup \mathcal{S} \times \mathcal{S}$ for which $\mathrm{decquote}(\mathrm{quote}(p) \parallel q) = (p, q)$ for all texts $p, q \in \mathcal{S}$. Both $\mathrm{decquote}(\mathrm{quote}(p) \parallel q)$ and $\mathrm{quote}(p)$ must be computable in time linear in $|p| + |q|$, the length of $p \parallel q$ (as usual, for $s \in \mathcal{S}$, $|s|$ denotes the length of s). Intuitively, quote is just an unambiguous way to quote one text inside another. With a usual alphabet, $\mathrm{quote}(s) = \mathrm{dec}(|s|) \parallel : \parallel s$ is suitable. (It is not hard to see that a suitable quotation scheme always exists for any alphabet with at least two elements). We call the pair $(\mathrm{quote}, \mathrm{decquote})$ with the above properties a *quotation scheme*. The software runtime uses quote and decquote, so both operations must be mechanical and efficient. The quotation scheme is

fixed independently of $(Q, \mathcal{T})$. The quotation scheme also makes Goal injective: the quoted answer and request can be recovered in order from $\mathrm{Goal}(x, y)$. Consequently, Eq. (2) is equivalent to

$$\mathrm{Goal}(x, y) \in \mathcal{T} \quad \Longleftrightarrow \quad (x, y) \in Q \qquad \text{for every } x, y \in \mathcal{S}. \tag{3}$$

With this we are ready to define the *reliability* and *completeness* of a system. For these definitions we view systems by the partial maps that their behavior defines. Let $F : \mathcal{S} \rightharpoonup \mathcal{S}$ be such a map. Reliability is defined relative to Q, while completeness will be defined relative to a set $D \subseteq \mathrm{dom}(Q)$, which contains those strings for which we expect the system to give an answer.

Fix an independently specified target family $D \subseteq \mathrm{dom}(Q)$.

Definition 1.1. A system whose behavior is described by a map $F : \mathcal{S} \rightharpoonup \mathcal{S}$ is called *reliable* relative to Q if every answer it returns on a request in $\mathrm{dom}(Q)$ is correct:

$$\mathrm{graph}\big(F|_{\mathrm{dom}(Q)}\big) \subseteq Q\,. \tag{4}$$

Furthermore, we say that the system is *D-complete* for some set $D \subseteq \mathrm{dom}(Q)$ if the domain of F includes D:

$$D \subseteq \mathrm{dom}(F)\,. \tag{5}$$

Reliability constrains the answers returned on requests in $\mathrm{dom}(Q)$; behavior outside $\mathrm{dom}(Q)$ is not part of the specification. If recognizing all (or some) such inputs is itself part of the task, choose a superset $U \subseteq \mathcal{S}$ of $\mathrm{dom}(Q)$ for which such responses are desired, choose a distinguished answer $\perp \in \mathcal{S}$ and replace Q by

$$Q^{\perp} = Q \cup \{(x, \perp) : x \in U \setminus \mathrm{dom}(Q)\} \tag{6}$$

while enlarging $\mathcal{T}$ accordingly so that the compatibility condition Eq. (2) still holds. Then $\mathrm{dom}(Q^{\perp}) = U$, and returning $\perp$ on an otherwise meaningless request becomes an ordinary correctness requirement.

Reliability alone still permits a system that never answers. Completeness requires the system to answer every request in D.

The input format affects completeness rather than reliability. A bounded request can appear verbatim in an input to the checking model. For a request of unbounded length, completeness requires a uniform way to establish assertions about the complete request from bounded local checks. The runtime theorem states the required coverage property, and Section 8 gives several ways to establish it using generic string operations.

2 Certificates before runtimes

The first result concerns finite derivations and does not depend on a learned model or a runtime. A derivation begins with true assertions and applies rules that preserve membership in $\mathcal{T}$. If its final assertion is $\mathrm{Goal}(x, y)$, the compatibility condition in Eq. (2) makes y a correct answer to x. This section defines the textual rules and certificates needed to state that argument precisely.

A rule contains text with named positions at which other texts may be inserted. Let ϵ denote the empty text, and let $\mathsf{H} = \mathcal{S} \setminus \{\epsilon\}$ be the set of *hole names*. A *template* is a finite expression built from literal texts, named holes, concatenation, and quotation. Let Tpl denote the set of all templates. A *substitution* assigns ordinary texts to hole names. Evaluating a template under a substitution produces an ordinary text.

Table 1: Template forms and the recursive definition of Inst

Template form	Value under a substitution σ
$\mathrm{text}(s)$, $s \in \mathcal{S}$	$\mathrm{Inst}(\mathrm{text}(s), \sigma) = s$
$\mathrm{var}(a)$, a a hole name	$\mathrm{Inst}(\mathrm{var}(a), \sigma) = \sigma(a)$
$\mathrm{join}(t_1, \ldots, t_k)$, $k \geq 0$	$\mathrm{Inst}(\mathrm{join}(t_1, \ldots, t_k), \sigma) = \mathrm{Inst}(t_1, \sigma) \parallel \cdots \parallel \mathrm{Inst}(t_k, \sigma)$
$\mathrm{quote}(t)$	$\mathrm{Inst}(\mathrm{quote}(t), \sigma) = \mathrm{quote}(\mathrm{Inst}(t, \sigma))$

Templates are built recursively using the four forms in Table 1. In the last two rows, $t, t_1, \ldots, t_k$ are templates already built, so join and quote may be nested to any finite depth. The function

$$\mathrm{holes} : \mathsf{Tpl} \longrightarrow \{A \subseteq \mathsf{H} : A \text{ is finite}\}$$

maps a template to the set of its hole names. For a finite list of templates $\mathbf{t} = (t_1, \ldots, t_m)$, define

$$\mathrm{holes}(\mathbf{t}) := \bigcup_{i=1}^{m} \mathrm{holes}(t_i), \qquad \mathrm{Sub}(\mathbf{t}) := \{\sigma : \mathrm{holes}(\mathbf{t}) \to \mathcal{S}\}.$$

Thus every $\sigma \in \mathrm{Sub}(\mathbf{t})$ has a finite domain and assigns a text to every hole occurring in the list. For each t_i in the list and $\sigma \in \mathrm{Sub}(\mathbf{t})$, Table 1 defines $\mathrm{Inst}(t_i, \sigma) \in \mathcal{S}$.

Definition 2.1 (Textual rule)**.** For $k \in \mathbb{N}$, a *textual rule* $r = (p_1, \ldots, p_k; c)$ consists of premise templates $p_1, \ldots, p_k \in \mathsf{Tpl}$ and one conclusion template $c \in \mathsf{Tpl}$. Define $\mathrm{Sub}(r) := \mathrm{Sub}((p_1, \ldots, p_k, c))$. The rule is *sound relative to* $\mathcal{T}$, or $\mathcal{T}$*-sound*, if

$$\forall \sigma \in \mathrm{Sub}(r), \quad [\ \forall i \in \{1, \ldots, k\},\ \mathrm{Inst}(p_i, \sigma) \in \mathcal{T}\ ] \Longrightarrow \mathrm{Inst}(c, \sigma) \in \mathcal{T}. \tag{7}$$

For $k = 0$, every instantiated conclusion must belong to $\mathcal{T}$.

When $\mathcal{T}$ is fixed, we use *sound* as shorthand for $\mathcal{T}$-sound.

The set of textual rules is therefore

$$\mathsf{Rules} := \bigcup_{k \in \mathbb{N}} (\mathsf{Tpl}^k \times \mathsf{Tpl}).$$

We write an element of $\mathsf{Tpl}^k \times \mathsf{Tpl}$ as $(p_1, \ldots, p_k; c)$ to distinguish the premise list from the conclusion.

A rule with no holes and no premises represents one particular assertion, such as $1 + 1 = 2$. We call such an object a *direct statement*.

Definition 2.2 (Certificate over a library)**.** Let $\mathcal{L} \subseteq \mathsf{Rules}$ be a nonempty finite *rule library*. For $n \in \mathbb{N}$, write $[n] = \{1, \dots, n\}$, with $[0] = \varnothing$.

A *certificate over* $\mathcal{L}$ of length $m \geq 1$ is a sequence

$$d = \big(r_i, \sigma_i, \rho_i\big)_{i=1}^{m}.$$

For every $i \in [m]$, write $r_i = (p_{i,1}, \dots, p_{i,k_i}; c_i)$. The sequence must satisfy

$$r_i \in \mathcal{L}, \qquad \sigma_i \in \mathrm{Sub}(r_i), \qquad \rho_i : [k_i] \longrightarrow [i-1], \tag{8}$$

$$\mathrm{Inst}(p_{i,\ell}, \sigma_i) = \mathrm{Inst}(c_{\rho_i(\ell)}, \sigma_{\rho_i(\ell)}) \qquad \text{for every } \ell \in [k_i]. \tag{9}$$

The *conclusion* of d is

$$\mathrm{concl}(d) := \mathrm{Inst}(c_m, \sigma_m) \in \mathcal{S}. \tag{10}$$

The matching requirement Eq. (9) says that every instantiated premise must be the conclusion of the earlier application selected by ρ_i. An *answer certificate* for answer y to request x is a certificate d whose conclusion is $\mathrm{Goal}(x, y)$.

Theorem 2.3 (Sound certificates give correct answers)**.** *Let $\mathcal{L} \subseteq \mathsf{Rules}$ be a library of $\mathcal{T}$-sound rules. If d is an answer certificate over $\mathcal{L}$ for answer y to request x, then $(x, y) \in Q$.*

Proof. We prove by induction over the certificate that every instantiated conclusion belongs to $\mathcal{T}$. For the current application, every instantiated premise is the conclusion of an earlier application and therefore belongs to $\mathcal{T}$ by the induction hypothesis. The current rule is $\mathcal{T}$-sound, so its instantiated conclusion also belongs to $\mathcal{T}$. The argument includes rules with no premises. The final conclusion is $\mathrm{Goal}(x, y)$, so Eq. (3) gives $(x, y) \in Q$. □

The theorem identifies the entire semantic argument. The certificate may be arbitrarily long, and its instantiated claims may be arbitrarily long. Only the truth-preservation property of its rules and the exact premise matches matter.

Definition 2.4 (Finite-rule certifiability)**.** Fix $D \subseteq \mathrm{dom}(Q)$. The family D is *finitely rule-certifiable* relative to $(\mathcal{T}, \mathrm{Goal})$ if there is a nonempty finite library $\mathcal{L} \subseteq \mathsf{Rules}$ such that

(i) every rule in $\mathcal{L}$ is $\mathcal{T}$-sound; and

(ii) for every $x \in D$, some $y \in \mathcal{S}$ has an answer certificate over $\mathcal{L}$.

The correctness specification is *globally finitely rule-certifiable* when $D = \mathrm{dom}(Q)$.

Finite-rule certifiability is a coverage property, not an efficiency claim. It requires one finite rule library for the entire family, while allowing the certificate length and all substituted texts to depend on the request. Theorem 2.3 shows that the resulting answers are automatically correct.

3 The scope of finite textual rules

The certificate theorem is useful only if finite rule libraries can express substantial computations. This section relates the framework to classical deductive systems and explains the sense in which finite textual rules have general computational scope.

A textual rule with several premises is a general finitary inference-rule schema. It is not specifically modus ponens. If the assertion language has implication, the rule

$$p_1, \ldots, p_k; c$$

can be encoded by the curried assertion $p_1 \to (p_2 \to \cdots (p_k \to c) \cdots)$ and repeated applications of modus ponens. The present framework does not require those logical connectives. It directly represents the more general operation of deriving one instantiated text from finitely many earlier instantiated texts.

Finite systems of such operations are classical. Post canonical systems, unrestricted grammars, and finite string-rewriting systems use finitely many schemas to generate sets of strings [1, 2]. Their computational universality gives the following correspondence. The statement permits the assertion language to contain tagged descriptions of machine configurations in addition to goal claims.

Theorem 3.1 (Computational scope of finite rule libraries)**.** *For every recursively enumerable relation $Q \subseteq \mathcal{S} \times \mathcal{S}$ and the fixed computable injective goal map* Goal*, there are a compatible set of true assertions $\mathcal{T}$ and a finite $\mathcal{T}$-sound textual rule library whose answer certificates derive exactly the goal claims* $\mathrm{Goal}(x, y)$ *with $(x, y) \in Q$. Conversely, the goal claims derivable from any fixed effective finite textual rule library form a recursively enumerable set.*

Proof idea. Choose a Turing machine that enumerates Q. Encode its configurations as tagged texts. A finite collection of textual rules represents the initial configuration, each transition, and the conversion of an accepting configuration for (x, y) into $\mathrm{Goal}(x, y)$. A computation history is then a finite certificate. Let $\mathcal{T}$ contain exactly the valid configuration and goal claims generated by these rules. The tags keep intermediate claims outside the range of Goal, so compatibility holds. This is the standard simulation of computation by a finite production system.

For the converse, enumerate all finite strings as possible certificates and run the mechanical certificate checker on each one. Every derivable goal claim eventually appears, and no other goal claim appears. □

3.1 Relation to mechanical proof systems

Once a finite library has been established as $\mathcal{T}$-sound, checking a certificate over that library does not require deciding soundness again. The checker instantiates templates, compares premises, and reads the final conclusion. With an explicit certificate representation, these operations take time polynomial in the certificate size and can be implemented in time linear in the total amount of materialized text.

This is the setting of an abstract proof system. Cook and Reckhow define a propositional proof system as a polynomial-time computable map whose range is exactly the propositional tautologies [3]. Equivalently, a supplied proof can be checked in polynomial time, while soundness of the checker is established mathematically rather than tested by an oracle during each proof check. A finite $\mathcal{T}$-sound textual library similarly gives a polynomial-time checker for its derivable claims. If every correct goal claim has a certificate, it gives a proof system for the language $\mathrm{Goal}(Q)$. Our coverage requirement is weaker: finite-rule certifiability of D asks for at least one certified correct answer for every request in D, not certificates for every pair in Q.

The learned-verifier construction addresses a different step. Instead of placing a task-specific rule library in the trusted runtime, it lets the generator propose textual rules and asks a bounded verifier to authorize them. A finite authorized library can subsequently be cached or compiled into an ordinary mechanical checker.

The theorem gives an exact expressiveness statement after the representation of intermediate assertions may be chosen. The theorem does not establish that a pre-existing natural-language truth set admits convenient bounded rules. For the applications in this lecture, we use the following thesis: whenever humans can explain a correctness relation through finitely described local steps, those steps can be expressed by a finite library of bounded textual rules. The arithmetic and tax constructions test the thesis on concrete families. The thesis is comparable in status to a choice of computational model: the formal results begin after the finite rule library has been specified. The classical results behind the theorem are discussed in the bibliographical notes.

4 Implementing certificate checking with learned models

The semantic theorem assumes a $\mathcal{T}$-sound rule library. We now describe a generic runtime that accepts rules proposed by a generator and uses a learned verifier to decide which proposed rules may be used. The runtime itself only performs syntactic operations.

A *model call* supplies an input text to a learned model and receives an output text. Let $\mathbb{N} = \{0, 1, 2, \ldots\}$. A *generator* is a partial function

$$h : \mathcal{S} \times \mathbb{N} \rightharpoonup \mathcal{S}.$$

On attempt n, the runtime evaluates $h(x, n)$. The returned text is intended to encode a *candidate*: a proposed answer and a certificate. A *verifier* is a partial function

$$g : \mathcal{S} \rightharpoonup \{0, 1\}.$$

Its input is a *checking question*. The generator and verifier may use different learned models or the same learned model in different calls.

The complete data flow is as follows.

1. The runtime receives a request x and calls the generator.
2. The generator proposes an answer y and a finite sequence of rule applications.
3. For each proposed rule, the runtime constructs a checking question and calls the verifier. For each application, the runtime instantiates the rule, compares every premise verbatim with an earlier conclusion, and stores the new conclusion after all checks pass.
4. The runtime returns y only if the final stored conclusion is exactly $\mathrm{Goal}(x, y)$. It may request another candidate after a rejection.

The generator performs the open-ended search. The verifier judges bounded semantic questions about proposed rules. The runtime parses records, resolves substitutions, checks references and exact string equality, and controls the attempts. Generation and verification may be interleaved; the sequential description only makes the checker and its proof easier to state.

For example, for $x =$ `What is 1+1?` and $y =$ `2`, one candidate contains the direct statement $\mathrm{Goal}(x, y)$. If the verifier accepts that statement, the runtime stores it, compares it with the mechanically constructed $\mathrm{Goal}(x, y)$, and returns y.

A proposed rule may be accompanied by a *justification* text $j \in \mathcal{S}$; $j = \epsilon$ means that none is supplied. Define

$$\mathrm{Ask} : \mathsf{Rules} \times \mathcal{S} \longrightarrow \mathcal{S}$$

so that $\mathrm{Ask}(r, j)$ is the complete checking question for rule r and justification j. If r has a premise or a hole, the question states the universal truth-preservation condition in Eq. (7) and displays the rule and justification. For a direct statement, the question asks whether its conclusion is a supported true assertion. A justification helps the verifier assess the rule; the runtime never treats it as an established premise. Appendix A gives the reversible English rendering.

For the direct statement with conclusion `1 + 1 = 2` and empty justification, $\mathrm{Ask}(r, \epsilon)$ consists of the following instruction and statement:

> Is the following statement a supported true assertion? Return 1 to certify the statement, or 0 to withhold certification. Ill-defined or unsupported statements are not supported true assertions. Any accompanying justification may be used to assess the statement.

````
Statement:
```
1 + 1 = 2
```
````

A total *target map* $v : \mathcal{S} \to \{0, 1\}$ specifies the desired verifier judgments. Malformed or semantically undefined questions receive value zero. The semantic requirement on v is

$$\forall (r, j) \in \mathsf{Rules} \times \mathcal{S}, \quad v(\mathrm{Ask}(r, j)) = 1 \quad \Longrightarrow \quad r \text{ is } \mathcal{T}\text{-sound.} \tag{11}$$

The implication is deliberately one-way: the target may reject some $\mathcal{T}$-sound rules. Reliability needs soundness of every accepted rule; coverage separately needs acceptance of enough rules.

For a finite rule library $\mathcal{L}$, an *annotation* is a map $\mathcal{J} : \mathcal{L} \to \mathcal{S}$ assigning a justification to every rule. We say that $(\mathcal{L}, \mathcal{J})$ is *accepted by* v if

$$v(\mathrm{Ask}(r, \mathcal{J}(r))) = 1 \qquad \text{for every } r \in \mathcal{L}. \tag{12}$$

Once a finite $\mathcal{T}$-sound library is known, no oracle is needed to recognize that particular library. The target map

$$v_{\mathcal{L},\mathcal{J}}(w) = \begin{cases} 1, & w = \mathrm{Ask}(r, \mathcal{J}(r)) \text{ for some } r \in \mathcal{L}, \\ 0, & \text{otherwise} \end{cases} \tag{13}$$

is sound and can be implemented in time linear in $|w|$ by exact comparison with a fixed finite table. The learned verifier is useful because it can authorize rules from their bounded textual descriptions instead of requiring every task library to be built into the runtime.

5 The runtime algorithm

Section 2 defined the mathematical content of a certificate. This section gives the exact data supplied to the runtime and the algorithm that checks those data.

A candidate has two fields: an answer $y \in \mathcal{S}$ and a finite list of records. Each record has four fields:

Field	Type and purpose
`rule`	Either a direct statement or the premise and conclusion templates of a rule.
`bindings`	Exactly one supplied text or interval reference for each hole.
`premises`	An ordered list of earlier record indices, one per premise.
`justification`	Optional justification text; the empty string is permitted.

We fix an unambiguous string encoding of this finite data. Encoding, parsing, and validating the declared field types are mechanical operations. A string that does not encode the required two fields and record fields is malformed and is rejected.

A *binding* is either a literal text or an *interval reference* (q, a, z). The *source selector* q is the tag `input`, the tag `answer`, or an earlier record index. For a request x, proposed answer y, and array A of earlier claims, these selectors denote x, y, and the selected entry of A, respectively. If u is the selected text and $0 \le a \le z \le |u|$, then $u[a : z]$ is the referenced text. The pseudocode operation `Resolve` replaces every interval reference by this substring and returns the resulting substitution. It rejects unavailable sources, invalid endpoints, and binding lists that do not supply exactly one value for every hole in the rule.

The checker is parameterized by a *query bound* $b \in \mathbb{N}$. A candidate that requires a checking question longer than b is rejected. The procedure `CHECK` below receives a request x, an encoded candidate, the verifier g, and b. It either rejects the candidate or accepts its proposed answer. The array A, initially empty, stores the claims established by accepted records.

```
CHECK(x, candidate, g, b):
    parse candidate as (answer y, finite record list records)
    if the encoding, declared fields, or field types are invalid: reject
    A := empty array
    for (r, bindings, refs, j) in records, where r = (p_1,...,p_k; c_r):
        require refs contains k valid earlier record indices
        sigma := Resolve(bindings, x, y, A, r)
        for ell := 1,...,k:
            require Inst(p_ell, sigma) == A[refs[ell]]
        w := Ask(r, j)                  # complete checking question
        require |w| <= b
        require g(w) == 1
        append Inst(c_r, sigma) to A
    require A is nonempty and A[-1] == Goal(x, y)
    accept y
```

Every failed requirement rejects the candidate. The verifier receives only w, the complete general rule and optional justification. The bindings, request, answer, record indices, and array A remain in runtime memory. This separation permits a bounded verifier question even when an instantiated claim is long.

The resulting system $R_b^{g,h}$ tries $n = 0, 1, 2, \ldots$ in order, runs `CHECK` on each returned candidate, and returns the first accepted answer.

Connecting the certificate to the answer. The last comparison identifies which request and answer the certificate addresses. The preceding rule applications must already have established the truth of $\mathrm{Goal}(x, y)$.[1]

A direct judgment of $\mathrm{Goal}(x, y)$ includes the actual request and answer in the verifier query. Replacing the request with a hole changes the judgment into a claim about *every* text assigned to that hole. For example, a rule asserting without premises that "Yes" correctly answers an arbitrary request is unsound. A useful general finishing rule instead has premises establishing what the request asks and why the answer satisfies it. Those premises must themselves be established earlier in the certificate. The runtime can manipulate arbitrarily long texts; a successful certificate must justify their use through bounded rule queries.

6 The runtime theorem

The certificate theorem gives the semantic guarantee. The runtime theorem shows that bounded verifier reliability and mechanical record processing enforce its hypotheses. The theorem also turns finite-rule certifiability into completeness when the verifier accepts an annotation of the covering library.

Theorem 6.1 (Reliability, preservation of certificates, and search)**.** *Fix Q, $\mathcal{T}$,* Goal*, and the quotation conventions satisfying Eq. (2), and fix a target v satisfying Eq. (11). For an integer $b \geq 0$, suppose*

$$\forall w \in \mathcal{S}, \quad |w| \leq b \implies g(w) \text{ terminates and } g(w) = v(w). \tag{14}$$

Then the runtime has the following properties.

(i) ***Reliability for every generator.*** *For every partial generator h, $\mathrm{graph}(R_b^{g,h}|_{\mathrm{dom}(Q)}) \subseteq Q$.*

(ii) ***Mechanical preservation.*** *Let $\mathcal{L} \subseteq$ Rules be a nonempty finite library, and let $\mathcal{J} : \mathcal{L} \to \mathcal{S}$ be an annotation accepted by v. If*

$$b \;\geq\; b_{\mathcal{L},\mathcal{J}} := \max_{r \in \mathcal{L}} |\,\mathrm{Ask}(r, \mathcal{J}(r))|, \tag{15}$$

there is a mechanical encoder $E_{\mathcal{J}}$ that, given x, y, and any answer certificate d over $\mathcal{L}$ for (x, y), produces a candidate accepted by `CHECK`*. Encoding requires no new semantic inference or proof search.*

(iii) ***Completeness by search.*** *Suppose D is finitely rule-certifiable through $\mathcal{L}$, the annotation $\mathcal{J}$ is accepted by v, and $b \geq b_{\mathcal{L},\mathcal{J}}$. One computable generator h_{enum} then makes $R_b^{g,h_{\mathrm{enum}}}$ complete on D.*

The bound $b_{\mathcal{L},\mathcal{J}}$ depends on the fixed annotated library and complete query format. The bound does not depend on the length of the request, answer, substitutions, references, or certificate.

Proof. For part (i), suppose the runtime returns y on $x \in \mathrm{dom}(Q)$. Every accepted record has a rule r and justification j for which $g(\mathrm{Ask}(r, j)) = 1$. The query bound and Eq. (14) give $v(\mathrm{Ask}(r, j)) = 1$, so Eq. (11) makes every used rule $\mathcal{T}$-sound. The runtime's exact comparisons make the record sequence a certificate, and its final comparison makes the sequence an answer certificate for (x, y). Theorem 2.3 gives $(x, y) \in Q$.

For part (ii), traverse the applications of d in their given order. For each application copy its rule and $\mathcal{J}(r)$ into a record, copy its substitution as literal bindings, and copy its premise indices. Serialize the answer and records in the fixed candidate format. Every exact premise comparison passes by the definition of a certificate. Each query is one of the fixed library queries and has length at most b. The acceptance condition Eq. (12) and Eq. (14) make every model verdict equal to one. The last conclusion is $\mathrm{Goal}(x, y)$, as required.

For part (iii), fix an order on Σ and enumerate $\mathcal{S}$ by length, breaking ties lexicographically. On attempt n, h_{enum} returns the nth text. Every finite text has a finite position in this enumeration. Parsing and all mechanical operations terminate on finite candidates; every model call made by `CHECK` terminates by Eq. (14). For each $x \in D$, finite-rule certifiability supplies y, d, and part (ii) supplies a finite accepted encoding $E_{\mathcal{J}}(x, y, d)$. The enumeration eventually reaches that encoding, unless an earlier accepted answer has already been returned. Either event gives termination; part (i) gives correctness. □

Corollary 6.2 (A finite learning target with unbounded uses)**.** *For a fixed annotated library $(\mathcal{L}, \mathcal{J})$, the competence requirement concerns only finitely many possible texts of length at most $b_{\mathcal{L},\mathcal{J}}$. Subject to that requirement, the runtime's reliability guarantee covers every accepted certificate, with no bound on its length or the lengths of its instances.*

The corollary follows from the finiteness of Σ and Theorem 6.1. Once the verifier is reliable on the bounded queries, the runtime extends that reliability to long certificates by storing and manipulating their instances in external memory.

Corollary 6.3 (Finite-rule certifiability is sufficient)**.** *If D is finitely rule-certifiable, there are a finite query bound b and a sound target map v for which the runtime is reliable for every generator and complete on D for one computable generator.*

Proof. Let $\mathcal{L}$ be a finite $\mathcal{T}$-sound library certifying D, choose any annotation $\mathcal{J}$, and use the finite-table target $v_{\mathcal{L},\mathcal{J}}$ from Eq. (13). Set $b = b_{\mathcal{L},\mathcal{J}}$. The claims follow from parts (i) and (iii). □

This corollary makes the existence of the required bounded verifier explicit. The finite-table target is not a practical learning theorem. It shows that the semantic coverage assumption leaves only a finite collection of bounded judgments to be implemented reliably.

7 Short requests: meaning, reasoning, and calculation

Interpreting a request establishes what an answer must satisfy. Reasoning or calculation then establishes that a proposed answer satisfies those conditions. In this section, the request wording fits in a checking query. The examples will identify the judgments that use this assumption. Section 8 replaces those judgments when the request is too long for one verifier call.

The grade-school example first shows the complete mechanism using two direct statements and one implication. The later examples introduce shared logical and computational rules when repeated composition makes those rules useful. Predicate names abbreviate English assertion templates; the checking questions contain their full text. All examples use empty justification and assume Eq. (14) with a sufficient query bound for their libraries.

7.1 A grade-school arithmetic question

The request x is "What do you get if you multiply the sum of 17 and 25 by 3?" Because the complete problem fits in one bounded call, the generator could propose the answer `126` and the single direct statement $\mathrm{Goal}(x, \texttt{126})$. If the verifier accepts that statement, the runtime's final comparison accepts the answer. Such a certificate uses the verifier to solve the complete short problem.

A three-record certificate exposes the reasoning without decomposing the arithmetic further. The generator proposes the following two direct statements:

1. The request "What do you get if you multiply the sum of 17 and 25 by 3?" asks for the value, written as a canonical decimal numeral, of the arithmetic expression `((17 + 25) * 3)`.

2. The arithmetic expression `((17 + 25) * 3)` has the value represented by the canonical decimal numeral `126`.

The verifier receives each statement exactly as displayed, together with the fixed instructions for a direct statement. Both records have empty justification: the verifier may recognize the interpretation and calculation without a supplied derivation.

The third record must connect the two accepted statements to the returned answer. The generator therefore supplies one reusable implication, with text holes x, e, y:

$$\frac{\begin{array}{c}\text{request } x \text{ asks for the value of arithmetic expression } e \text{ as a canonical decimal numeral} \\ \text{arithmetic expression } e \text{ has the value represented by canonical decimal numeral } y\end{array}}{\mathrm{Goal}(x, y)}. \tag{16}$$

The verifier checks this universally quantified implication as a rule over arbitrary texts in the three holes. For the present record, the generator binds x to the exact input, e to `((17 + 25) * 3)`, and y to the exact proposed answer `126`. The runtime instantiates both premises and compares them verbatim with records 1 and 2. The runtime then constructs the conclusion and compares it with $\mathrm{Goal}(x, \texttt{126})$. Thus the verifier checks the two semantic judgments and the general implication; the runtime checks their particular use and the connection to the original request and returned answer.

In the runtime's record format, the certificate is:

Record	Object sent to verifier	Bindings	Premise records
0	first direct statement	none	none
1	second direct statement	none	none
2	implication in Eq. (16)	$x = x_{\text{input}}$, $e =$ `((17+25)*3)`, $y = y_{\text{answer}}$	0, 1

Here x_{input} and y_{answer} are interval references to the complete stored request and proposed answer. The verifier receives no bindings or record indices. Those objects remain in runtime memory.

The two direct statements and the implication use empty justification. Justification would be optional text supplied to help the verifier judge one statement or rule; it would not become an additional premise.

Three failures illustrate the division of responsibility. If the first statement assigns the wrong expression to the request, the verifier rejects that statement. If the second statement gives the wrong value, the verifier rejects the calculation. If the generator returns `127` after establishing the value `126`, the runtime cannot match the second premise of Eq. (16) with the proposed answer and rejects the candidate.

7.2 Reusable logical rules

The generator can separate interpreting the request from reasoning with its facts and rules. For context text $C \in \mathcal{S}$ and statement text $p \in \mathcal{S}$, write $H(C, p)$ for the assertion text

> Under the rules and facts in "{{C}}", the following Boolean statement is well-defined and true:
> "{{p}}"

The placeholders hold the context and statement texts. The assertion says that p is meaningful in context C and follows from its rules and facts. Write $W(C, p)$ for the assertion that p is a well-defined Boolean statement in that context.

We use $p \wedge q$, $p \vee q$, and $p \leftrightarrow q$ for text constructors. For example, $p \wedge q$ produces "Both of the following statements hold:" followed by quote(p) and quote(q) on separate lines. The other constructors say "at least one" and "are equivalent." Both constituent statements must be meaningful in the same context; the connectives have their usual propositional meanings. The following five schemas are therefore sound textual rules:

$$\frac{H(C,p) \quad H(C,q)}{H(C, p \wedge q)}, \qquad \frac{H(C, p \wedge q)}{H(C,p)}, \qquad \frac{H(C, p \wedge q)}{H(C,q)}, \tag{17}$$

$$\frac{H(C,p) \quad W(C,q)}{H(C, p \vee q)}, \qquad \frac{H(C, p \leftrightarrow q) \quad H(C,q)}{H(C,p)}. \tag{18}$$

Conjunction preserves both established statements; disjunction needs one established branch and a meaningful second branch. The last rule substitutes an equivalent statement. In every rule, the same context is used throughout.

The verifier checks these general schemas. The generator supplies the actual C, p, q as bindings, and the runtime checks the instantiated premises against earlier claims. Consequently a long context can occur in every stored claim without being repeated in every model query.

For self-contained assertions, conjunction introduction has the form

$$\frac{p \quad q}{p \wedge q}. \tag{19}$$

We use this version for the desk instructions.

Here is the actual rule-specific part of the contextual conjunction query. Double braces mark text placeholders, and the quotation marks specify how the inserted texts are quoted. The common checking instructions appear in Appendix A.

> Premise 1:
> ```
> Under the rules and facts in “{{C}}”, the following Boolean statement is well-defined and true:
> “{{P}}”
> ```
>
> Premise 2:
> ```
> Under the rules and facts in “{{C}}”, the following Boolean statement is well-defined and true:
> “{{Q}}”
> ```
>
> Conclusion:
> ```
> Under the rules and facts in “{{C}}”, the following Boolean statement is well-defined and true:
> “Both of the following statements hold:
> ‘{{P}}’
> ‘{{Q}}’”
> ```

7.3 Writing forty desk instructions

The original request is:

> Give me 40 single-sentence instructions for tidying a desk. Each instruction must describe a concrete action, be understandable without the other instructions, and contain at most twelve words. Count words by splitting on whitespace. Put one instruction on each line.

Call this request x_{desk}. The generator first proposes sufficient conditions for an answer. For request text x, let $\mathrm{Spec}(x)$ denote the assertion

> Every text consisting of exactly forty newline-terminated lines, each line, excluding its terminating newline, consisting of one sentence with at most twelve whitespace-separated words, describing a concrete action for tidying a desk, and understandable without the other lines, correctly answers the request “{{x}}”.

The first certificate record asks the verifier to judge $\mathrm{Spec}(x_{\mathrm{desk}})$ directly. The complete original request is quoted in that query. The proposed plan distinguishes conditions on each entry from the required count and layout of the whole output.

The generator then proposes forty lines, including "Place loose pens in a pen holder." The entry checks separate two assertions about each entry s:

$\mathrm{Form}(s)$**.**
The text $\mathrm{quote}(s)$ contains no newline and consists of one sentence with at most twelve whitespace-separated words.

$\mathrm{Content}(s)$**.**
The text $\mathrm{quote}(s)$ describes a concrete action for tidying a desk and is understandable without other instructions.

A direct judgment establishes each assertion. Conjunction introduction then gives $\mathrm{Good}(s)$, where

$$\mathrm{Good}(s) = \mathrm{Form}(s) \wedge \mathrm{Content}(s). \tag{20}$$

Thus the verifier can judge the format and meaning separately. The runtime uses the same entry text in both checks and in their conjunction.

For the first instruction, the rule-specific part of the format query is:

> Statement:
> ```
> The text "Place loose pens in a pen holder." contains no newline and consists of one sentence
> with at most twelve whitespace-separated words.
> ```

For list processing, $\mathrm{All}(L, n)$ says that n contains only bars, L has exactly $|n|$ newline-terminated entries, and every entry satisfies Good. Here $|n|$ counts the bars. The generator uses

$$\frac{}{\mathrm{All}(\epsilon, \epsilon)}, \qquad \frac{\mathrm{All}(L, n) \quad \mathrm{Good}(s)}{\mathrm{All}(L \parallel s \parallel \texttt{\textbackslash n}, n \parallel \texttt{|})}, \tag{21}$$

$$\frac{\mathrm{Spec}(x) \quad \mathrm{All}(L, \texttt{|}^{40})}{\mathrm{Goal}(x, L)}, \tag{22}$$

where ϵ denotes the emptry string. The notation `\n` denotes one newline. Appending one entry preserves the line boundaries because $\mathrm{Form}(s)$ excludes newlines. The last rule uses the previously checked specification of the request. The generator supplies an interval reference to the original request for x; the runtime checks that the instantiated $\mathrm{Spec}(x)$ matches the first claim. Its final comparison checks the exact returned list.

Proposition 7.1 (Desk-answer reliability and coverage)**.** *Assume the soundness and competence hypotheses of Theorem 6.1 with query bound b.*

(i) If y violates any requirement in x_{desk}, `CHECK` rejects every finite candidate with answer y.

(ii) Fix a finite nonempty set $\mathcal{A} \subseteq \mathcal{S}$ of suitable individual instructions. Suppose v accepts the direct judgment of $\mathrm{Spec}(x_{\mathrm{desk}})$*, the ordinary conjunction rule, the three displayed list rules, and the direct judgments of* $\mathrm{Form}(s)$ *and* $\mathrm{Content}(s)$ *for $s \in \mathcal{A}$, and their complete queries have length at most b. Every forty-entry sequence from $\mathcal{A}$ has an accepted certificate for x_{desk}.*

Proof. For part (i), the induction in Part (i) puts every accepted claim in $\mathcal{T}$. The final comparison would therefore establish $\mathrm{Goal}(x_{\mathrm{desk}}, y) \in \mathcal{T}$, contradicting the violated requirement. All checks on a finite candidate terminate under the competence hypothesis, so the candidate is rejected.

For part (ii), establish $\mathrm{Spec}(x_{\mathrm{desk}})$ and apply the empty-list rule. For each entry use two direct judgments, conjunction introduction, and the append rule. The finishing rule establishes $\mathrm{Goal}(x_{\mathrm{desk}}, L)$. Part (ii) encodes the certificate. There are $1 + 1 + 4 \cdot 40 + 1 = 163$ records. □

The two compositions have different roles: propositional reasoning combines requirements on one entry, and list induction extends the guarantee to the complete output. For instance, $\mathcal{A}$ may contain every suitable instruction below a fixed byte limit; the implementation uses forty listed instructions.

A rejected candidate. Suppose the generator supplies only the first thirty-nine instructions. Every entry passes its checks, and the append rules establish $\mathrm{All}(L, \mathsf{I}^{39})$. The finishing rule requires $\mathrm{All}(L, \mathsf{I}^{40})$. The runtime rejects the candidate when those texts fail to match; it does not ask the verifier to count the long list.

7.4 The simple tax request

Here is the complete original input:

> An applicant meets the standard qualification exactly when they are resident and have income below the limit.
>
> An applicant is eligible exactly when both conditions hold: they meet the standard qualification or are a qualifying student; and they have not received benefit B. These are the complete eligibility conditions.
>
> I am resident, my income is below the limit, and I have not received benefit B. I do not know whether I am a qualifying student.
>
> Am I eligible under these rules?

Write x_{tax} for this text and y_{tax} for "Yes, you are eligible." Set $U(s) = H(x_{\mathrm{tax}}, s)$. The statement texts used in the certificate are complete sentences:

Name	Text
R	The applicant is resident.
L	The applicant has income below the limit.
Q_s	The applicant meets the standard qualification.
S	The applicant is a qualifying student.
N	The applicant has not received benefit B.
E	The applicant is eligible.

The generator supplies four judgments interpreting the input, six logical steps, and the final connection to the answer:

Record	Conclusion	Justification
0	$U((R \wedge L) \wedge N)$	supplied applicant facts
1	$U(Q_s \leftrightarrow (R \wedge L))$	first paragraph
2	$U(E \leftrightarrow ((Q_s \vee S) \wedge N))$	second paragraph
3	$W(x_{\text{tax}}, S)$	meaningful applicant statement
4	$U(R \wedge L)$	0, conjunction elimination
5	$U(N)$	0, conjunction elimination
6	$U(Q_s)$	1 and 4, equivalence
7	$U(Q_s \vee S)$	6 and 3, disjunction
8	$U((Q_s \vee S) \wedge N)$	7 and 5, conjunction
9	$U(E)$	2 and 8, equivalence
10	$\text{Goal}(x_{\text{tax}}, y_{\text{tax}})$	9, answer to the stated question

Records 0–3 are generator-proposed assertions interpreting the supplied facts and rules. The verifier judges each assertion with the full short request quoted in its query. Records 4–9 use the general schemas from Section 7.2: the model sees a context hole, while the runtime substitutes x_{tax}. Record 10 uses a rule connecting eligibility to this particular question and answer. The unknown student status needs no truth judgment because the other branch of the disjunction is established.

A rejected candidate. Keep all eleven records but propose the answer "No, you are ineligible." The records still establish $\text{Goal}(x_{\text{tax}}, \texttt{Yes, you are eligible.})$. The final comparison rejects the candidate because that claim names a different answer. A valid certificate cannot be attached to an arbitrary returned response.

7.5 Where the short-request constructions stop working

The three constructions use bounded general rules for most of their reasoning, but each begins with a semantic judgment about concrete input text. The arithmetic certificate asks the verifier to judge what the complete question asks. The desk certificate asks it to judge $\text{Spec}(x_{\text{desk}})$, with the complete request quoted in the query. Records 0–3 of the tax certificate ask it to interpret the supplied rules and facts with the complete request quoted as their context. These checking questions grow when the corresponding source text grows.

The later uses of the interpreted text do not have this problem. The logical rules in Section 7.2 contain a context hole, so the verifier judges a bounded schema while the runtime substitutes the stored context. The desk-list rules similarly process an unbounded answer through repeated bounded schemas. The runtime's final comparison with $\text{Goal}(x, y)$ is mechanical and can involve an arbitrarily long x or y.

Consequently the short constructions fail first at the ground semantic judgments that interpret the actual request. If one of their complete queries exceeds b, `CHECK` rejects the candidate. The next section shows how the generator can replace one large interpretation judgment by bounded rules whose applications account for the complete input.

8 Accounting for complete long inputs

The runtime theorem applies to a request of any finite length, but its completeness conclusion assumes that a certificate of the answer exists. This section begins with an arithmetic expression whose complete interpretation may exceed the verifier-input bound. The example shows how a generator can replace that judgment by bounded rules and a certificate. We then define the general property used by the construction. The runtime does not contain a parser for each task; it checks the proposed organization through exact operations on strings.

8.1 Arithmetic expressions without a runtime parser

An arithmetic expression is presented as an ordinary input string. The runtime has no arithmetic parser. The generator may instead propose a parsing derivation using finitely many grammar schemas. For example, schemas can say that a decimal numeral is an expression and that suitably delimited sums and products of expressions are expressions. Each conclusion constructs the parent text from the child texts. A derivation whose root is the exact input therefore certifies the proposed parse.

The generator can also account for the expression through a reduction sequence. Consider the request

```
What is the value of (3+4)*5?
```

A certificate may establish the following claims:

$$3 + 4 = 7,$$
$$(3 + 4) * 5 \text{ reduces in one value-preserving step to } 7 * 5,$$
$$7 * 5 = 35,$$
$$7 * 5 \text{ has value } 35,$$
$$(3 + 4) * 5 \text{ has value } 35,$$
$$\text{Goal}(\texttt{What is the value of (3+4)*5?}, 35).$$

The first and third claims are bounded arithmetic judgments. The other claims follow from bounded schemas for reducing a complete subexpression, composing reduction with evaluation, and answering the displayed form of question. The rule for a reduction step must express when the replaced interval is a complete subexpression; this condition may follow from delimiters, from earlier syntactic claims, or from a finite collection of valid evaluation contexts. The runtime checks only that the bound intervals, surrounding text, and reconstructed expressions match exactly.

The same schemas can process an expression of arbitrary length. Expression length increases the number of records and the lengths of their instantiated claims, but does not increase the lengths of the schemas submitted to the verifier. If the individual numerals are also arbitrarily long, their arithmetic requires a further certificate. The next construction provides one for addition.

8.2 Coverage rules

Define the textual constructor Account : $\mathcal{S} \times \mathcal{S} \to \mathcal{S}$ by

$$\text{Account}(x, m) = \texttt{The complete text} \parallel \text{quote}(x) \parallel \texttt{has summary} \parallel \text{quote}(m) \parallel . \tag{23}$$

The quotation scheme makes the exact source text x and summary text m recoverable from $\text{Account}(x, m)$. A summary may state a syntactic category, an expression value, the state obtained after processing a list of provisions, or the dependencies of a section.

Definition 8.1 (Compositionally accountable family)**.** A family $D \subseteq \mathrm{dom}(Q)$ is *compositionally accountable relative to* $\mathcal{T}$ if there is a nonempty finite rule library $\mathcal{L}_{\mathrm{account}}$ such that

(i) every rule in $\mathcal{L}_{\mathrm{account}}$ is $\mathcal{T}$-sound; and

(ii) for every $x \in D$, there are $m \in \mathcal{S}$ and a certificate d over $\mathcal{L}_{\mathrm{account}}$ satisfying $\mathrm{concl}(d) = \mathrm{Account}(x, m)$.

The definition states the exact property needed from the input-handling part of a certificate. The property is relative to $\mathcal{T}$ because the summaries and composition rules carry semantic content. It does not require a particular input syntax. Intervals, record labels, list positions, parse trees, and dependency graphs are ways to construct the required certificates rather than parts of the definition.

One common rule has the form

$$\frac{\mathrm{Account}(z_1, m_1) \quad \cdots \quad \mathrm{Account}(z_j, m_j)}{\mathrm{Account}(R_c(z_1, \ldots, z_j), M_c(m_1, \ldots, m_j))}. \tag{24}$$

The display abbreviates an ordinary textual rule: R_c and M_c are templates formed from literal text, holes, concatenation, and quotation. The conclusion therefore contains the exact text constructed from the child texts. The generator may propose a tree of applications of such rules. The verifier checks the bounded rule schemas, while the runtime substitutes the possibly long child texts and checks every premise and concatenation. A root claim $\mathrm{Account}(x, m)$ accounts for the exact request x.

The same runtime supports several organizations of a long input.

- *Exact reconstruction* combines intervals or records until their concatenation is the complete input.
- A *parsing derivation* uses finitely many grammar rules to build a root claim about the complete input. The generator supplies the derivation; the runtime has no task-specific parser.
- A *reduction sequence* starts with the exact input and applies approved local replacements until it reaches a form from which the answer is immediate.
- A *sequential certificate* processes every record in order while carrying a summary state. Consecutive indices and the final index certify that no record was omitted.
- A *dependency certificate* follows labelled references and proves that the selected records are closed under the dependency relation supplied by the input format.

These methods use only the runtime's generic operations on strings, substitutions, and references. They differ in the rules proposed by the generator.

Coverage does not by itself establish semantic relevance. A concatenation tree can show that every character occurs without showing that the resulting summary is correct. The verifier checks the semantic content of the proposed rules, including the rules that say when a summary supports an answer. The certificate demonstrates that those rules work together for the particular request. The runtime checks their composition and the connection to the exact input and proposed answer.

8.3 A general lifting result

The next theorem separates the formal coverage condition from any particular way of satisfying it.

Theorem 8.2 (From input accounting to answers)**.** *Suppose D is compositionally accountable through $\mathcal{L}_{\text{account}}$. Suppose there is a finite library $\mathcal{L}_{\text{solve}}$ of $\mathcal{T}$-sound rules such that, for every $x \in D$, at least one witness (m, d) to Definition 8.1 can be extended as follows: there are $y \in \mathcal{S}$ and a finite sequence of applications of rules in $\mathcal{L}_{\text{solve}}$ in which every premise is either* $\text{Account}(x, m)$ *or the conclusion of an earlier application, and the final conclusion is* $\text{Goal}(x, y)$*. Then D is finitely rule-certifiable through $\mathcal{L} = \mathcal{L}_{\text{account}} \cup \mathcal{L}_{\text{solve}}$.*

Proof. Fix $x \in D$. Choose the witness (m, d) and append its solution sequence to d, using the established account claim as an available premise. The resulting certificate ends with $\text{Goal}(x, y)$ and uses only rules in the finite $\mathcal{T}$-sound library $\mathcal{L}$. This is finite-rule certifiability. □

If the target map accepts annotations of both libraries, their union has one finite query bound and Theorem 6.1 gives runtime completeness on D. The runtime does not receive the libraries as trusted task definitions: rules occur in records proposed by the generator and are submitted to the verifier.

8.4 The informal generality claim

The theorem begins after the bounded rules and certificates have been specified. The connection to human-understandable organizations of text is an informal thesis rather than a mathematical theorem.

> **Compositional verification thesis.** If a human can make clear how complete inputs in a family determine answers using semantic rules whose complete verifier questions have one fixed length bound, together with mechanically checkable bookkeeping that accounts for the complete input, then a sufficiently capable generator can express that method as rules and certificates for the generic runtime.

The verifier checks the proposed bounded rules that collectively describe how the parts are processed and when the accumulated information permits an answer. A certificate for a particular request uses those rules to produce an answer, and the runtime checks every use by string manipulation. The fixed length bound also makes the set of possible checking questions finite. The phrase "a human can make clear" has no formal definition, so the thesis cannot be proved from the definitions above. The role of the thesis is analogous to the role of the informal notion of effective computation in the Church–Turing thesis. Once the required bounded rules and certificates are given, Theorems 6.1 and 8.2 provide the formal reliability and completeness conclusions.

9 Worked constructions for long inputs

This section gives complete accounting and solution certificates for two families. The first construction adds arbitrarily long decimal integers. The second processes tax requests containing an unbounded exclusion list and an unbounded ordered list of provisions.

9.1 Adding integers of arbitrary length

Consider requests in the following format:

```
Add the following two equal-width strings of decimal digits.
Return exactly one more digit than either input, using an initial 0 if necessary.
```

```
First: DIGITS
Second: DIGITS
```

The two occurrences of `DIGITS` have the same positive length. Each contains only the characters `0`,...,`9`. The required leading output digit makes the output length fixed by the input format. A mechanical preprocessor can convert an ordinary pair of nonnegative decimal numerals to this format by padding the shorter numeral with leading zeros; it can remove the extra leading zero from the returned answer afterward.

The generator treats each operand as a sequence of digit characters. For digit characters $a, b, d \in \{\mathtt{0}, \dots, \mathtt{9}\}$ and carry characters $c, c' \in \{\mathtt{0}, \mathtt{1}\}$, let $\mathrm{Digit}(a, b, c; d, c')$ be the assertion

$$\mathrm{value}(a) + \mathrm{value}(b) + \mathrm{value}(c) = \mathrm{value}(d) + 10\,\mathrm{value}(c'). \tag{25}$$

There are only 200 true digit transitions, one for every choice of a, b, c. Each transition is a bounded direct statement.

For equal-length digit strings u, v, w and $c \in \{\mathtt{0}, \mathtt{1}\}$, let $\mathrm{Add}(u, v, w, c)$ assert that right-to-left column addition of u and v, starting with carry zero at the right edge, produces the digit string w and carry c at the left edge. The base and composition rules are

$$\frac{}{\mathrm{Add}(\epsilon, \epsilon, \epsilon, \mathtt{0})}, \qquad \frac{\mathrm{Add}(u, v, w, c) \quad \mathrm{Digit}(a, b, c; d, c')}{\mathrm{Add}(a \parallel u, b \parallel v, d \parallel w, c')}. \tag{26}$$

The generator processes the columns from right to left. At each step, the runtime checks that the new digits are the characters immediately preceding the already processed suffixes, matches the carry to the preceding claim, and constructs the longer strings by concatenation. The verifier checks the fixed composition rule and the applicable one-digit transition.

Let $X(u, v)$ denote the complete displayed request with operands u, v. After all columns have been processed, the finishing rule is

$$\frac{\mathrm{Add}(u, v, w, c)}{\mathrm{Goal}(X(u, v), c \parallel w)}. \tag{27}$$

The conclusion contains the complete reconstructed request and the proposed output. The runtime therefore checks that every input digit was used, in the correct position, and that the final claim concerns the exact original request and returned answer.

Proposition 9.1 (Uniform verification of decimal addition)**.** *There is a finite library of $\mathcal{T}$-sound textual rules through which the family of displayed addition requests is finitely rule-certifiable. For operands of length n, the certificate uses $2n + 2$ records. If the target map accepts an annotation of the library, one query bound suffices for every operand length; under Theorem 6.1, no incorrect sum is returned.*

Proof. The library contains the base rule, the composition and finishing rules, and the 200 true instances of Eq. (25). The library is finite, so any accepted annotation gives a finite query bound through Eq. (15).

Start with the base claim. For each of the n columns, use one digit-transition record and one application of the composition rule. Induction on the number of processed columns proves that the resulting Add claim contains the correct result suffix and outgoing carry. One finishing record gives the goal, for $1 + 2n + 1 = 2n + 2$ records. The runtime theorem gives rejection of every incorrect returned answer. □

Other arithmetic operations require different local summaries. Schoolbook multiplication, for example, can record partial products and accumulated rows. A formal proof system has the same general shape at a higher level: formulas have a compositional syntax, a proof is a sequence of formula records, and a fixed collection of inference rules determines which record may follow from which earlier records. A runtime can check substitutions, references, and exact formula construction while a verifier checks the bounded inference-rule schemas. This observation indicates how a fixed formal logic can be covered; the details depend on the chosen syntax and proof calculus.

9.2 A structured tax input: exclusions and eligibility

This example uses the sequential coverage method from Section 8. The input marks definitions, additional exclusions, case facts, and the question. The generator proposes rules for processing these blocks and a certificate for the particular case. The runtime checks that the certificate reconstructs and processes the exact blocks in the input. The number of exclusion clauses may be arbitrarily large. Here is one complete instance, copied from the executable example:

> Apply only the complete fictional code supplied below. All case statements describe the same applicant. Quoted flag names denote exact strings. Part E lists all additional exclusions. The applicant is excluded under Part E if any listed clause applies. Every statement in the Case block is supplied as a fact. The Definitions block applies throughout this request.
>
> Definitions:
>
> Section 10. The applicant is eligible for credit C exactly when the applicant is resident under section 40, has assessable income under section 112 below 20000, has not received benefit B, and is not excluded under section 950 or Part E.
>
> Section 40. The applicant is resident exactly when the applicant has at least 183 days of residence.
>
> Section 112. Assessable income is earnings minus the deduction under section 113.
>
> Section 113. The deduction is the smaller of approved training expenses and 500.
>
> Section 950. An applicant younger than 18 is excluded.
>
> End definitions.
>
> Part E:
>
> An applicant with disqualification flag "overseas-credit" is excluded.
>
> An applicant with disqualification flag "duplicate-award" is excluded.
>
> An applicant with disqualification flag "late-filing" is excluded.
>
> End Part E.
>
> Case:
>
> The applicant has 190 days of residence, earnings of 20300, approved training expenses of 700, and age 23. The applicant has not received benefit B.
>
> Disqualification flag "overseas-credit" is false.
>
> Disqualification flag "duplicate-award" is false.
>
> Disqualification flag "late-filing" is false.
>
> End case.
>
> Question: Is the applicant eligible for credit C?

The Definitions block applies throughout the request. Part E collects alternative reasons for exclusion: any applicable exclusion is sufficient. The Case block supplies all its statements jointly as facts. The closing markers delimit the blocks; quoted flag names remain data even if they contain words such as “End Part E.”

The generator proposes a certificate that accounts for every exclusion and its corresponding case fact. It then specifies the complete eligibility condition and evaluates it. These operations use the same runtime as the short examples.

Write $C_\beta(d, w, t, a)$ for the case-text constructor, with numeral texts for residence days, earnings, training expenses, and age. The index $\beta \in \{0, 1\}$ selects "has not received" or "has received" benefit B. Let $X_\beta(E, F; d, w, t, a)$ construct the complete request using exclusion text E, matching flag-fact text F, and that case paragraph. Its header, definition block, opening and closing markers, question, and line breaks are fixed as displayed. The generator proposes intervals for E and F in the original request. The runtime copies those intervals verbatim; the rules below establish what their contents imply.

For this family, the generator treats the request as fixed definitions, a recursive exclusion list, case facts, and a question. The constructors X_β, e, and f below build the corresponding exact texts. The block markers and quotation convention let the runtime check the proposed decomposition by equality and concatenation.

9.2.1 Accounting for the long part of the input

For any name text $s \in \mathcal{S}$, define

$$e(s) = \texttt{An applicant with disqualification flag } \| \operatorname{quote}(s) \| \texttt{ is excluded.\textbackslash n},$$
$$f(s) = \texttt{Disqualification flag } \| \operatorname{quote}(s) \| \texttt{ is false.\textbackslash n}.$$

A name can itself be arbitrarily long. Quotation keeps newlines and other characters inside the name. For a finite list $s_1, \ldots, s_m$, put

$$E = e(s_1) \,\|\, \cdots \,\|\, e(s_m), \qquad F = f(s_1) \,\|\, \cdots \,\|\, f(s_m). \tag{28}$$

The assertion $\mathrm{Pair}(u, v)$ says that u, v are exactly one exclusion line and its false-flag fact with the same quoted name. The assertion $\mathrm{Exclusions}(E, F)$ says that the two texts consist entirely of such lines, paired in order. Consequently every exclusion in E is inactive under the supplied facts F.

This is the list construction already used for desk instructions, applied to paired lines:

$$\frac{}{\mathrm{Exclusions}(\epsilon, \epsilon)}, \qquad \frac{}{\mathrm{Pair}(e(s), f(s))}, \tag{29}$$

$$\frac{\mathrm{Exclusions}(E, F) \quad \mathrm{Pair}(u, v)}{\mathrm{Exclusions}(E \,\|\, u, F \,\|\, v)}. \tag{30}$$

After m append steps, the stored claim covers the complete two lists. The verifier has seen the same three rule schemas regardless of list length or name length; the generator supplies the long bindings and the runtime checks every concatenation and premise reference. In the terminology of Section 8, these rules account for the two lists by exact reconstruction and sequential processing.

9.2.2 Deriving and evaluating the eligibility condition

The tax calculation uses a fixed expression tree whose leaves are short numerals and Boolean constants. Let $\mathrm{Num}(a)$ assert that a is a canonical decimal numeral, and let $\mathrm{Eval}(e, z)$ assert that the closed expression text e has the value represented by z. For an operation $\circ$, let $\mathrm{Calc}_\circ(a, b, c)$ assert that applying $\circ$ to the values represented by a, b gives the value represented by c. If $\mathrm{op}_\circ(A, B)$ constructs the corresponding expression text, the reusable evaluation rule is

$$\frac{\mathrm{Eval}(A, a) \quad \mathrm{Eval}(B, b) \quad \mathrm{Calc}_\circ(a, b, c)}{\mathrm{Eval}(\mathrm{op}_\circ(A, B), c)}. \tag{31}$$

A numeral is evaluated by $\mathrm{Num}(a) \Rightarrow \mathrm{Eval}(a, a)$. Each scalar calculation in this example fits in one bounded direct statement.

Define $\mathrm{BoolQ}(x, e)$ to assert that e is a well-formed closed Boolean expression and the correct answers to x are related to its value by

$$(x, \texttt{Yes.}) \in Q \iff e \text{ evaluates to true}, \qquad (x, \texttt{No.}) \in Q \iff e \text{ evaluates to false}. \tag{32}$$

Following the code's references yields the expression constructor

$$B_\beta(d, w, t, a) = (d \geq 183) \wedge \big((w - \min(t, 500) < 20000) \wedge (\neg\beta \wedge (a \geq 18))\big). \tag{33}$$

The display describes expression text: d, w, t, a occupy numeral positions, and $\neg\beta$ is written as the literal `true` or `false`. The generator proposes the following rule specifying the answer condition for each β:

$$\frac{\mathrm{Num}(d),\ \mathrm{Num}(w),\ \mathrm{Num}(t),\ \mathrm{Num}(a),\ \mathrm{Exclusions}(E, F)}{\mathrm{BoolQ}(X_\beta(E, F; d, w, t, a), B_\beta(d, w, t, a))}. \tag{34}$$

The fixed sections appear literally in this rule's request template. The rule follows section 40 to the residence test, sections 112 and 113 to the income expression, and section 950 to the age test. Its last premise accounts for every additional exclusion. The verifier checks that these conditions are equivalent to eligibility under the complete supplied code. The rule leaves the numerical tests and conjunction to be evaluated.

For the displayed case, the generator now supplies the following calculations:

Calculation	Result
$190 \geq 183$	true
$\min(700, 500)$	500
$20300 - 500$	19800
$19800 < 20000$	true
$23 \geq 18$	true

Each scalar calculation is a short direct judgment. The evaluator in Eq. (31) combines them into the value of the complete expression. Conjunction uses the four rows of its usual Boolean truth table. Here all four eligibility conditions are true, so the result is true. Replacing age 23 by 17 makes the age comparison false; replacing earnings 20300 by 20500 makes the income comparison false. Receiving benefit B makes the corresponding Boolean constant false. The same evaluation rules then produce false and the answer "No."

The two finishing rules are

$$\frac{\mathrm{BoolQ}(x, e) \quad \mathrm{Eval}(e, \texttt{true})}{\mathrm{Goal}(x, \texttt{Yes.})}, \qquad \frac{\mathrm{BoolQ}(x, e) \quad \mathrm{Eval}(e, \texttt{false})}{\mathrm{Goal}(x, \texttt{No.})}. \tag{35}$$

The runtime first matches the complete exclusion and flag texts to the established Exclusions claim. It later matches the evaluated expression to the answer-condition claim, then compares the final claim with the original request and returned answer. If the generator certifies fewer exclusions than the input contains, using the full input block fails the premise match; using the shorter certified block describes a different request and fails the final match. Changing a definition, a block marker, or a case fact also invalidates the old certificate through these exact matches.

A rejected candidate with an omitted exception. Change the last case fact to "Disqualification flag `"late-filing"` is true." The applicant is now excluded. Suppose the generator returns "Yes." after certifying only the first two exclusion–fact pairs. Its answer-condition record proposes intervals for the complete three-clause Part E and all three flag facts. The required Exclusions(E, F) premise differs from the established two-pair claim, so the runtime rejects that record. No verifier call needs to contain the complete exclusion list.

9.2.3 Coverage of the family

Fix an integer $K \geq 0$. Let D_K consist of the requests $X_\beta(E, F; d, w, t, a)$ with the following data:

- d, w, t, a are canonical decimal numerals for integers in $\{0, \ldots, K\}$;
- $\beta \in \{0, 1\}$;
- (E, F) is constructed by Eq. (28) from any finite list of name texts.

The number of exclusions and the lengths of their names are unbounded.

Proposition 9.2 (Variable cases and unbounded source text)**.** *For every fixed K, there is a finite library $\mathcal{L}_K$ of sound rules such that every request in D_K has a certificate for its correct answer using at most $2m + 36$ rule applications, where m is the number of additional exclusions. Thus D_K is finitely rule-certifiable. If the target map accepts an annotation of $\mathcal{L}_K$, one query bound works for every request in D_K.*

Proof. Include the three list rules, the two answer-condition rules, the required instances of the evaluation schema, numeral evaluation, and the two finishing rules. Include direct judgments for input numerals from 0 through K, the fixed constants $18, 183, 500, 20000$, the Boolean constants, and every scalar calculation arising from these inputs. There are finitely many such calculations: the case tuple ranges over a finite set, and the expression has fixed size. Subtraction judgments include negative assessable incomes. All primitive judgments can be checked by integer arithmetic and Boolean truth tables.

The pair and append rules preserve the exact line pairing. The answer-condition rules follow by substituting the definitions in the fixed code and observing that each additional exclusion has a false flag. The evaluation rule preserves expression values under each typed operation. The finishing rules follow from the definition of BoolQ. Thus every included rule is sound.

For a request in D_K, construct its exclusion summary in $2m + 1$ steps. Use at most four input-numeral judgments and one step establishing the answer condition. The expression has at most eight distinct numeral leaves, of which at most four still need numeral judgments; evaluating the leaves and the one Boolean constant therefore takes at most thirteen further steps. Its eight binary operations take two steps each: a scalar calculation judgment and an application of Eq. (31). One finishing step gives the goal. The total is at most $2m + 1 + 4 + 1 + 13 + 16 + 1 = 2m + 36$.

For any annotation $\mathcal{J}_K$ of $\mathcal{L}_K$ accepted by v, Theorem 6.1 supplies the encoding and the bound $b_{\mathcal{L}_K, \mathcal{J}_K}$. The library is fixed once K is fixed; increasing the source's list or name lengths changes only the runtime's bindings and number of records. □

A generator can recover the parameters from the displayed input syntax, or enumerate their possible values until the constructed text matches the request. It then builds the certificate of the answer condition and its evaluation just described.

9.2.4 When the tax code itself is long

The preceding construction allows an unbounded exclusion list, but its fixed definitions still fit in one verifier query. A long rule block needs its own compositional format. Consider a Part A consisting of an ordered list of two kinds of provisions:

> Deduct an allowance of n, capped at the remaining income.
>
> Cap the remaining income at n.

Here n is a decimal numeral. The surrounding code says to start from the applicant's earnings and apply every provision once, in source order. The generator proposes a sequential treatment of Part A as provision records. The runtime checks that each extension appends the exact next provision text. The generator must account for every record because the answer-condition rule later uses the complete Part A interval copied from the input.

For Part A text A and numeral texts w, z, let $\mathrm{Run}(A, w, z)$ assert that executing exactly the provisions in A, starting from income w, leaves income z. The empty sequence and the two possible extensions use

$$\frac{\mathrm{Num}(w)}{\mathrm{Run}(\epsilon, w, w)}, \tag{36}$$

$$\frac{\begin{array}{cc}\mathrm{Run}(A, w, z) & \mathrm{Num}(n) \\ \mathrm{Calc}_{\min}(z, n, h) & \mathrm{Calc}_{-}(z, h, r)\end{array}}{\mathrm{Run}(A \parallel \mathrm{clause}(n), w, r)}, \qquad \frac{\mathrm{Run}(A, w, z) \quad \mathrm{Num}(n) \quad \mathrm{Calc}_{\min}(z, n, r)}{\mathrm{Run}(A \parallel \mathrm{cap}(n), w, r)}. \tag{37}$$

The verifier checks these fixed rules. The runtime matches the prefix claim and scalar calculations, appends the exact next provision text, and stores the new prefix claim. Induction therefore gives one claim covering the complete ordered block.

The order is semantically relevant. Starting from 20300, a cap at 20000 followed by a deduction of 1 leaves 19999, while the reverse order leaves 20000. The two inputs therefore have different correct eligibility answers. One certificate cannot certify the other order because its final Run claim contains different source text.

Let $X^*_\beta(A, E, F; d, w, a)$ construct the complete request with Part A text A, exclusion text E, matching flag facts F, case numerals d, w, a, and benefit status β. If z is the final income established by $\mathrm{Run}(A, w, z)$, define

$$B^*_\beta(d, z, a) = (d \geq 183) \wedge \big((z < 20000) \wedge (\neg\beta \wedge (a \geq 18))\big). \tag{38}$$

The answer-condition rule is

$$\frac{\mathrm{Run}(A, w, z) \quad \mathrm{Exclusions}(E, F) \quad \mathrm{Num}(d) \quad \mathrm{Num}(a)}{\mathrm{BoolQ}(X^*_\beta(A, E, F; d, w, a), B^*_\beta(d, z, a))}. \tag{39}$$

The same A occurs in the premise and the reconstructed request, and the same w occurs in the execution claim and the case paragraph. Consequently the runtime verifies that the calculation starts from the supplied earnings and covers the complete supplied provision block. The existing Boolean evaluation and finishing rules then establish the answer.

For example, 301 deductions of 1 from earnings of 20300 leave 19999; 300 such deductions leave 20000. The two requests have opposite answers. A certificate covering only 300 provisions cannot answer the 301-provision request because its Run claim fails the premise comparison in Eq. (39). Asking the verifier to judge the complete 301-provision request directly also fails when that query exceeds the bound.

Proposition 9.3 (Bounded checking of an unbounded tax code)**.** *Fix $K \geq 0$. Let D_K^* contain the requests above in which every case figure and provision amount belongs to $\{0, \ldots, K\}$, Part A is any finite ordered list of the two permitted provision forms, and the paired exclusion list is arbitrary. There is a finite sound rule library $\mathcal{L}_K^*$ that covers every request in D_K^*. A request with m provisions and ℓ exclusions has a certificate with at most $4m + 2\ell + 32$ records. One query bound works for every m, ℓ and every permitted request in the family.*

Proof. Every provision maps a balance in $\{0, \ldots, K\}$ to another balance in that set. The numeral and scalar-calculation statements needed by the two extension rules therefore form a finite set. Add those statements, the rules Eqs. (36) to (37), the exclusion rules, the answer-condition rules, and the Boolean evaluation and finishing rules. The resulting library is finite.

The base claim starts the execution certificate. Each provision needs at most one numeral statement, two scalar calculations, and one extension step. The exclusion list uses $2\ell+1$ records. The remaining case, Boolean-evaluation, and finishing records fit within the stated constant. Induction over Part A proves the complete Run claim; the earlier exclusion induction covers Part E. Theorem 6.1 supplies the uniform query bound and rejection of every incorrect returned answer. □

A general tax code may use named sections with references rather than one ordered list. A suitable long format can expose a labelled dependency graph: each section is a bounded record, and each reference names another record. A certificate must then account for every section on which the answer depends and justify the summary passed across each dependency edge. The sequence construction above is the special case in which the dependency graph is a path. The appropriate summary assertion depends on the rule language, so we do not develop a universal tax-law semantics here.

10 Implementation and further development

`runtime.py` contains the generic checker, certificate encoder, search loop, and exhaustive generator. The untrusted constructors in `examples.py` share the templates in `example_rules.py`. For reproducible tests, the verifier is a lookup table accepting 778 designated complete questions. Exact traces contain the original inputs, candidates, accepted claims, and question identifiers linked to the full question catalogue.

Example	Input bytes	Records	Maximum query bytes
Forty desk instructions	267	163	2,346
Simple tax	485	11	2,500
Arithmetic, three records	58	3	1,646
Arithmetic	58	12	3,340
Arithmetic, alternate wording	45	14	3,340
Full tax, three exclusions	1,541	42	4,103
Full tax, 512 exclusions	55,750	1,060	4,103
Full tax, one long flag name	18,897	38	4,103
Tax code with 301 allowances	18,671	934	4,405
Cap, then deduction	1,306	36	4,405
Deduction, then cap	1,306	34	4,405

The full-tax traces include the eligible case and negative answers caused by age, income, or benefit status. The allowance-code traces include both 301 and 300 provisions, giving opposite answers,

and the two orders of a cap and deduction. Trace milestones identify the claims about complete input blocks, answer conditions, and calculated results. The long-name case has one name of 8,810 characters. The scale examples use exactly the same verifier table and bound as the short examples. Tests cover the source-to-answer connections, shared logic, typed evaluation, and protocol rejection cases, including omitted premises, wrong answers, changed source text, omitted exclusions, invalid bindings, and malformed records. Additional tests check that query rendering preserves the rules and that generated TeX excerpts and saved query files match the executable templates.

The bounded-reliability assumption identifies a learning target rather than a practical learning theorem. For a fixed finite alphabet and bound, the set of possible verifier inputs is finite but may be astronomically large, and we may have no feasible procedure for establishing that a trained verifier is correct on every one of them. The target is nevertheless strictly weaker than requiring a learned system to answer every unrestricted problem correctly. The runtime supplies the length generalization: correctness on bounded verifier inputs implies correctness of every accepted answer, even when the request, answer, certificate, and computation have unbounded length.

The next coverage result should identify a substantial fragment of ordinary mathematical text, give a finite sound rule library, and construct certificates of the answers in that fragment. First-order derivability offers a concrete benchmark.

Search efficiency is the next algorithmic problem. Enumeration can be extremely expensive. Better generation, reuse of accepted judgments, and interleaving search with verification can reduce these costs.

11 Take-home messages

- Every answer certificate built from $\mathcal{T}$-sound rules certifies a correct answer. This semantic fact is independent of any runtime or learned model.
- A runtime can enforce the certificate conditions by using a bounded verifier to authorize textual rules and checking their applications mechanically.
- A generator can propose bounded rules that organize a long request. The runtime uses exact string operations to check that their applications account for the complete request.
- A finite $\mathcal{T}$-sound library can express any recursively enumerable correctness relation after suitable intermediate assertions are chosen. An accepted annotation of the library gives a uniform model-input bound and completeness by exhaustive certificate search.

Endnotes

1. Copying costs. The literal representation constructs claims containing the complete request and answer. A generator can refer to the original input by an interval, but the runtime still copies that text when instantiating the claim. Repeated expansion of intermediate claims can be expensive. Shared text representations could avoid repeated copies while preserving exact matching; such an implementation would need to preserve the instantiation and equality operations used in the proof.

Bibliographical notes

The semantic certificate layer is classical. Post's canonical systems use a finite set of production schemas over strings [1]; even restricted tag systems can simulate Turing machines [2]. Cook and Reckhow's abstract definition identifies a proof system with a polynomial-time map whose range is exactly the target language [3]. These results support Theorem 3.1 and the interpretation of a trusted finite library as a mechanical proof system.

Proof-carrying code has the same producer–certificate–checker separation: an untrusted producer supplies code and a safety proof, and the consumer runs a small checker [4]. Necula and Schneck subsequently allowed the producer to supply untrusted high-level proof rules together with evidence that the rules enforce the trusted safety policy [5]. Foundational proof certificates seek a common checker for evidence produced by different proof systems [6]. These systems use formal safety policies or logics. The learned-verifier layer in this note instead asks a bounded model call to judge whether a textual rule preserves membership in the intended natural-language truth set.

Lamport's account of structured proofs emphasizes named steps and explicit dependencies [11]. Machine checking of formal mathematical texts was a goal of de Bruijn's Automath project [12]. Metamath provides a concrete reference for checking substitutions against previously established statements [13]. The present runtime applies the same mechanical operation to textual rule schemas.

The long-input constructions draw on two treatments of structured text. SBVR separates shared vocabulary from rules, interprets separately stated rules jointly, and requires exceptions to be accommodated explicitly [14, Sections 13, 16.3–16.4]. Naproche distinguishes sentence syntax from the structure of definitions, assumptions, proofs, and other blocks; block boundaries control scope [15, Section 7.2]. We use these ideas for explicit scopes, references, and exceptions while allowing the generator to propose the rules that organize a particular input. Neither source supplies the runtime theorem or uniform query bound here.

Modern AI work supplies the closest motivation for the learned-verifier layer. Cobbe et al. generate many mathematical solutions and train a verifier to rank them [7]. Irving, Christiano, and Amodei propose debate so that agents reduce a difficult judgment to short statements a human can assess [8]. Brown-Cohen, Irving, and Piliouras prove that doubly efficient debate can verify polynomial-time computations with a constant number of human-judgment queries, including a setting in which a model supplies a solution and a natural-language justification [9]. Kirchner et al. train provers to produce solutions that small learned verifiers and time-limited humans can assess more reliably [10]. These methods study empirical verifier accuracy or interactive protocols. The construction here is noninteractive: a generator supplies a finite certificate, a bounded verifier authorizes its textual rules, and a deterministic runtime checks every application and the final request–answer claim.

Glossary

Correctness relation Q.
The externally specified relation between requests and acceptable answers.

Supported true assertion, $\mathcal{T}$.
A text whose agreed, context-explicit meaning is supported true, independently of a particular certificate.

Template.
An element of Tpl: a finite expression for literal text, a hole, concatenation, or quotation. Its

instances are ordinary texts of unrestricted length.

$\mathcal{T}$-sound textual rule.
A universally quantified implication preserving membership in $\mathcal{T}$ for every string assignment to its holes.

Direct statement.
A textual rule with no premises and no holes, presented to the verifier as one particular assertion.

Certificate.
An ordered list of rule applications with specified substitutions and earlier-premise references.

Verifier.
The system component that returns a binary judgment on a complete checking question. Its input–output map is g; its calls may use the same learned model as the generator.

Rule library $\mathcal{L}$.
A fixed finite set of textual rules.

Annotation $\mathcal{J}$.
A map assigning a justification text to every rule in a library. The target map accepts an annotation when it accepts every resulting checking question.

Finite-rule certifiability.
Existence of one finite $\mathcal{T}$-sound library that supplies an answer certificate for every request in the target family.

Answer-condition assertion $\mathrm{BoolQ}(x, e)$**.**
Equivalence between answering a yes–no request x and evaluating Boolean expression e.

Evaluation assertion $\mathrm{Eval}(e, z)$**.**
Expression e has the value represented by text z.

Prefix execution assertion $\mathrm{Run}(A, w, z)$**.**
Applying every deduction or cap provision in text A, starting from the amount represented by w, leaves the amount represented by z.

Completeness on D.
Eventual return of a correct answer on every request in the independently specified family D.

Compositionally accountable family.
A request family for which one finite $\mathcal{T}$-sound library can derive an $\mathrm{Account}(x, m)$ claim for every request x in the family.

Compositional verification thesis.
The informal claim that a human-clear method based on uniformly bounded semantic rules and mechanical coverage can be expressed as rules and certificates for the generic runtime.

A Exact certificate and query conventions

The executable implementation represents text as Unicode, encodes verifier queries in UTF-8, and measures their length in bytes. JSON string literals provide the concrete quotation scheme. These choices instantiate the abstract finite-string and quotation operations used in the main text.

The JSON constructors are the four arrays

```
["text",s]  ["var",name]  ["join",t1,...,tk]  ["quote",t].
```

A rule is an object with exactly the keys `premises` and `conclusion`. A candidate has exactly `answer` and `steps`. A record has exactly `rule`, `bindings`, `premises`, and `justification`. An interval binding has exactly `source`, `start`, and `end`. Sources are `"input"`, `"answer"`, or a nonnegative earlier claim index. Extra keys, duplicate JSON keys, invalid indices, incorrect binding domains, unsupported constructors, and invalid text encodings are rejected.

A.1 Constructing an English checking question

For an English-language verifier, the runtime constructs $\mathrm{Ask}(r, j)$ mechanically. It renders the premise and conclusion templates as English text with explicit placeholders, appends the optional justification, and prepends fixed English instructions. The runtime does not interpret the rule. For a rule with a premise or a hole, the implementation uses

$$\mathrm{Ask}(r, j) = \texttt{PROMPT} \parallel \texttt{"\textbackslash n\textbackslash n"} \parallel \mathrm{QueryBody}(r, j). \tag{40}$$

The fixed instructions are generated from `query_text.py`:

> Is the following rule sound? Return 1 to certify soundness, or 0 to withhold certification.
>
> Each placeholder {{name}} ranges over every finite text. Repeated occurrences of a name receive the same text. Substitution is simultaneous: inserted text is never read as further template notation.
>
> Curly quotation marks “...” and ‘...’ mark a quotation operation: substitute inside them, then write the result as a canonical JSON string literal, escaping non-ASCII characters. Nested quotations apply this operation from the inside out. Straight quotation marks are ordinary literal text. A backslash escapes a literal brace, curly quotation mark, or backslash. The escapes \n, \r, \t, \uHHHH and \UHHHHHHHH denote the corresponding characters; H is a hexadecimal digit. All other text and whitespace inside the fenced blocks is literal.
>
> For every assignment to the placeholders, if all premises are supported true assertions, the conclusion must also be a supported true assertion. With no premises, every instance of the conclusion must be supported true. Ill-defined or unsupported statements are not supported true assertions. Use the agreed meanings of the statements without adding task-specific assumptions. Any accompanying justification may be used to assess the rule.

For a direct statement, the runtime recognizes that the premise list and hole set are empty. It labels the conclusion as a statement and uses the shorter fixed instruction:

> Is the following statement a supported true assertion? Return 1 to certify the statement, or 0 to withhold certification. Ill-defined or unsupported statements are not supported true assertions. Any accompanying justification may be used to assess the statement.

The following excerpt is the executable implementation. The function `render_template`, defined earlier in the same file, performs the reversible template rendering described below.

```
    return not rule['premises'] and not _template_has_hole(rule['conclusion'])


def query_body(rule, justification=''):
    contents = [render_template(t) for t in rule['premises']]
    conclusion = render_template(rule['conclusion'])
    rendered_justification = escape_text(justification)
    runs = re.findall(r'`+', '\n'.join(contents + [conclusion, rendered_justification]))
    fence = '`' * max(3, 1 + max(map(len, runs), default=0))
    def block(label, text):
        return f'{label}:\n{fence}\n{text}\n{fence}'
    if is_direct_statement(rule):
        sections = [block('Statement', conclusion)]
    else:
        sections = [block(f'Premise {i}', t) for i, t in enumerate(contents, 1)]
        if not contents:
            sections.append('There are no premises.')
        sections.append(block('Conclusion', conclusion))
    if justification:
        sections.append(block('Justification', rendered_justification))
    return '\n\n'.join(sections)


def render_query(rule, justification=''):
    prompt = STATEMENT_PROMPT if is_direct_statement(rule) else PROMPT
    return prompt + '\n\n' + query_body(rule, justification)
```

In general, the rule body contains labeled premise and conclusion blocks and an optional justification block. A block is delimited by a line of backticks. The renderer chooses a delimiter longer than every backtick run in the displayed rule and justification, so literal content cannot close a block.

The renderer concatenates literal text, writes a hole named a as `{{a}}`, and shows a quotation operation with matched curly quotation marks. Successive nesting levels alternate double and single marks. Literal braces, curly quotation marks, and backslashes are escaped; control characters use the character escapes specified above. Ordinary newlines are preserved. These conventions distinguish a placeholder from literal text that happens to contain braces.

Proposition A.1 (The displayed rule preserves its meaning)**.** *Let* $\mathrm{Render} : \mathsf{Tpl} \to \mathcal{S}$ *map a template to its displayed text, and let* $\mathrm{Read} : \mathcal{S} \rightharpoonup \mathsf{Tpl}$ *decode displays that obey the conventions above. For every template* $t \in \mathsf{Tpl}$ *and assignment* σ *to its holes,*

$$\mathrm{Inst}(\mathrm{Read}(\mathrm{Render}(t)), \sigma) = \mathrm{Inst}(t, \sigma). \tag{41}$$

Consequently displaying a rule preserves the assertion that the verifier is asked to judge.

Proof. Character escaping preserves literal texts and hole names. Concatenation preserves their order; merging adjacent literal nodes or reassociating joins leaves instantiation unchanged. Matched quotation marks identify each quotation operation and its child. Induction on the template therefore gives the equation, including nested quotations. The fenced blocks preserve the premise order, conclusion, and justification separately. □

The runtime includes every byte of the common instructions, displayed rule, and justification in the query bound. Bindings and global references stay in runtime memory. Canonical quotation in stored claims still uses `json.dumps` with `ensure_ascii=True`; the readable display is a representation of the same template operations.

B Reproducing and inspecting the examples

In this directory, run

```
python3 examples.py
python3 -m unittest -v test_runtime.py
latexmk -pdf -interaction=nonstopmode -halt-on-error note.tex
```

The generator writes `traces/checking-questions.json`, a catalogue of the 778 questions accepted by the verifier fixture. Individual queries are also saved as ordinary text files in `traces/queries/`. Each example trace records its input, proposed answer and records, accepted claims, and the corresponding question identifiers. The files are:

- `desk.json`: the checked plan, forty-instruction answer, and 163 records;
- `simple-tax.json`: the four-paragraph request and eleven-record derivation;
- `arithmetic-simple.json`: the two direct statements and one reusable implication described in Section 7.1;
- `arithmetic.json`: twelve records, including exact source intervals;
- `arithmetic-rephrased.json`: fourteen records with a two-step reformulation;
- `full-tax-yes.json`, `full-tax-no.json`, `full-tax-benefit.json`, and `full-tax-income.json`: complete codes, case facts, and forty-two-record certificates for the four cases;
- `long-code-301.json` and `long-code-300.json`: opposite answers for codes that differ by one allowance provision;
- `cap-then-deduct.json` and `deduct-then-cap.json`: opposite answers caused by exchanging the order of two provisions;
- `statistics.json`: the measured sizes, including the long-input cases.

The directory `traces/rejected/` contains eight unsuccessful candidates: the missing desk entry, the opposite simple-tax answer, the two arithmetic errors, the omitted active exclusion, the omitted allowance, the reordered provisions, and the oversized direct query. Its `index.json` gives the rejection reason and failed record for each case. Each full trace includes the actual candidate, claims accepted before rejection, model queries and verdicts, and the exact unequal texts when a comparison fails. The oversized-query trace records the query that was not sent. These cases use the same checker, verifier fixture, and bound as the successful examples.

The files in `generated/` contain the exact query excerpts and measurement table included by this document. They are regenerated by the same command as the traces. A test compares every generated excerpt with its current code-derived content.

For example, the complete rule-specific part of the arithmetic answer-condition query is:

> Premise 1:
> ```
>
> The text “{{a}}” is the canonical decimal numeral for a nonnegative integer: either 0 or a digit from 1 through 9 followed by zero or more digits from 0 through 9.

```
Premise 2:
```

The text “{{b}}” is the canonical decimal numeral for a nonnegative integer: either 0 or a digit from 1 through 9 followed by zero or more digits from 0 through 9.

```
Premise 3:
```

The text “{{c}}” is the canonical decimal numeral for a nonnegative integer: either 0 or a digit from 1 through 9 followed by zero or more digits from 0 through 9.

```
Conclusion:
```

The request “What do you get if you multiply the sum of {{a}} and {{b}} by {{c}}?” asks a numerical question equivalent to finding the value of the well-formed closed arithmetic expression “(({{a}} + {{b}}) * {{c}})”, whose value is a nonnegative integer. For every canonical nonnegative decimal numeral n, n correctly answers the request if and only if it denotes that value.

```
```

The scalar addition judgment used later in that certificate is:

```
Statement:
```

The texts “17” and “25” are canonical decimal numerals for integers. Applying the usual + operation to the values they denote gives the value represented by “42”, which is a canonical decimal numeral for an integer.

```
```

The corresponding complete questions, including the common instructions, are `traces/queries/arithmetic-specification.txt` and `traces/queries/arithmetic-calculation.txt`. The three-record version uses the following complete questions:

- `traces/queries/arithmetic-simple-interpretation.txt`;
- `traces/queries/arithmetic-simple-calculation.txt`;
- `traces/queries/arithmetic-simple-composition.txt`.

References

[1] Emil L. Post. “Formal Reductions of the General Combinatorial Decision Problem.” *American Journal of Mathematics*, 65(2):197–215, 1943. https://doi.org/10.2307/2371809.

[2] John Cocke and Marvin Minsky. “Universality of Tag Systems with $P = 2$.” *Journal of the ACM*, 11(1):15–20, 1964. https://doi.org/10.1145/321203.321206.

[3] Stephen A. Cook and Robert A. Reckhow. “The Relative Efficiency of Propositional Proof Systems.” *Journal of Symbolic Logic*, 44(1):36–50, 1979. https://www.cs.toronto.edu/~sacook/homepage/cook_reckhow.pdf.

[4] George C. Necula. "Proof-Carrying Code." In *Proceedings of POPL*, pages 106–119, 1997. https://courses.grainger.illinois.edu/cs421/fa2010/papers/necula-pcc.pdf.

[5] George C. Necula and Robert R. Schneck. "Proof-Carrying Code with Untrusted Proof Rules." In *Proceedings of the International Software Security Symposium*, 2002. https://people.eecs.berkeley.edu/~necula/ISSS02/isss02.pdf.

[6] Dale Miller. *Foundational Proof Certificates: Making Proof Universal and Permanent*. LFMTP invited talk, 2013. https://www.lix.polytechnique.fr/Labo/Dale.Miller/papers/lfmtp13.pdf.

[7] Karl Cobbe et al. "Training Verifiers to Solve Math Word Problems." 2021. https://arxiv.org/abs/2110.14168.

[8] Geoffrey Irving, Paul Christiano, and Dario Amodei. "AI Safety via Debate." 2018. https://arxiv.org/abs/1805.00899.

[9] Jonah Brown-Cohen, Geoffrey Irving, and Georgios Piliouras. "Scalable AI Safety via Doubly-Efficient Debate." 2023. https://arxiv.org/abs/2311.14125.

[10] Jan Hendrik Kirchner, Yining Chen, Harri Edwards, Jan Leike, Nat McAleese, and Yuri Burda. "Prover-Verifier Games Improve Legibility of LLM Outputs." 2024. https://arxiv.org/abs/2407.13692.

[11] Leslie Lamport. *How to Write a 21st Century Proof*. 2011, revised 2012. https://lamport.azurewebsites.net/pubs/proof.pdf.

[12] N. G. de Bruijn. *Automath: A Language for Mathematics*. T.H.-Report 68-WSK-05, Technological University Eindhoven, 1968. https://pure.tue.nl/ws/portalfiles/portal/2039924/256169.pdf.

[13] Metamath contributors. *Metamath Proof Explorer*, especially "How Metamath Proofs Work." https://us.metamath.org/mpeuni/mmset.html. Accessed 30 October 2026.

[14] Object Management Group. *Semantics of Business Vocabulary and Business Rules*, version 1.5. October 2019. https://www.omg.org/spec/SBVR/1.5/PDF.

[15] Marcos Cramer. *Proof-checking mathematical texts in controlled natural language*. PhD thesis, University of Bonn, 2013. https://naproche-net.github.io/downloads/2013-doktor.pdf.